

- Description

This module manages a smart factory production line speed adjustment and equipment maintenance system, ensuring that:

1. **The production speed of manufacturing machines is continuously monitored and adjusted based on demand and efficiency.**
2. **If the production speed is too high, wear and tear on equipment is assessed, and maintenance warnings are issued.**
3. **If the speed is too low, efficiency is analyzed, and recommendations for increasing output are provided.**
4. **A log of all speed adjustments, maintenance warnings, and optimization actions is maintained for improving production reliability.**

This system helps balance efficiency, extend equipment lifespan, and optimize production throughput in a factory setting.

- Specification

module SmartFactoryProductionManagement

var currentSpeed, maxSpeed, minSpeed, optimalSpeed: real

var maintenanceWarning: boolean

var log: sequence of string

process updateSpeed(changeAmount: real)

functional scenario form:

currentSpeed $\geq$ minSpeed $\&\&$ log = log $\oplus$ ("Adjusting for Efficiency: " + currentSpeed + " units/hour."), Production speed updated successfully.

||

currentSpeed > maxSpeed $\&\&$ maintenanceWarning = true, log = log $\oplus$ ("Overproduction Warning: " + currentSpeed + " units/hour, Risk of Equipment Wear."), Overproduction warning triggered, maintenance needed.

||

currentSpeed < minSpeed $\&\&$ maintenanceWarning = false, log = log $\oplus$ ("Low Efficiency Alert: " + currentSpeed + " units/hour, Consider Increasing Output."), Low efficiency alert triggered, production adjustment recommended.

end process

end module

- Java Code

```
package code;
import java.util.*;
class ProductionLine {
    private double maxSpeed;
    private double minSpeed;
    private double optimalSpeed;
    private double currentSpeed;
    private boolean maintenanceWarning;
```

```

        private List<String> log;
        public ProductionLine(double maxSpeed, double minSpeed, double optimalSpeed, double initialSpeed) {
            this.maxSpeed = maxSpeed;
            this.minSpeed = minSpeed;
            this.optimalSpeed = optimalSpeed;
            this.currentSpeed = initialSpeed;
            this.maintenanceWarning = false;
            this.log = new ArrayList<>();
        }
        public void updateSpeed(double changeAmount) {
            currentSpeed += changeAmount;
            manageProductionEfficiency();
            String message = String.format("Current Speed: %.2f units/hour", currentSpeed);
            if (currentSpeed > maxSpeed) {
                message += " | WARNING: Overproduction! Risk of Equipment Wear.";
            } else if (currentSpeed < minSpeed) {
                message += " | ALERT: Low Efficiency! Consider Increasing Output.";
            } else if (currentSpeed > optimalSpeed) {
                message += " | INFO: Adjusting for Efficiency.";
            } else {
                message += " | Speed is Optimal.";
            }
            System.out.println(message);
            log.add(message);
        }
        private void manageProductionEfficiency() {
            if (currentSpeed > maxSpeed) {
                maintenanceWarning = true;
            } else {
                maintenanceWarning = false;
            }
        }
        public void printLog() {
            System.out.println("\nProduction Line Speed Log:");
            for (String logEntry : log) {
                System.out.println(logEntry);
            }
        }
    }
}

public class c50_SmartFactoryProduction {

```

```

    public static void main(String[] args) {
        ProductionLine productionLine = new ProductionLine(200.0, 50.0,
150.0, 120.0);
        // testcase-VT:
//      productionLine.updateSpeed(40.0);
//      productionLine.updateSpeed(-30.0);
//      productionLine.updateSpeed(-90.0);
//      productionLine.updateSpeed(100.0);
//      productionLine.updateSpeed(80.0);

        // testcase-VT:
//      productionLine.updateSpeed(0.0);
//      productionLine.updateSpeed(-1.7976931348623157E308);
//      productionLine.updateSpeed(5.992310449541052E307);
//      productionLine.updateSpeed(6.237000967295999E290);
//      productionLine.updateSpeed(-5.992310449541052E307);

        // testcase-Z3:
//      productionLine.updateSpeed(40.0);
//      productionLine.updateSpeed(-30.0);
//      productionLine.updateSpeed(-90.0);
//      productionLine.updateSpeed(100.0);
//      productionLine.updateSpeed(80.0);
//      productionLine.printLog();

        // testcase-UVT:
        productionLine.updateSpeed(40.0);
        productionLine.updateSpeed(-30.0);
        productionLine.updateSpeed(-90.0);
        productionLine.updateSpeed(100.0);
        productionLine.updateSpeed(80.0);
        productionLine.printLog();
    }
}

```

- Coverage report

1. Test case number: 5
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	1.2 %	10	832	842
▼ src	1.2 %	10	832	842
▼ code	1.2 %	10	832	842
▼ c50_SmartFactoryProduction.java	100.0 %	10	0	10
> c50_SmartFactoryProduction		0	0	0
> ProductionLine	100.0 %	10	0	10

3. Vibration testing instruction coverage: 97.9%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	0.7 %	143	21,344	21,487
✓ src	0.7 %	143	21,344	21,487
✓ code	0.7 %	143	21,344	21,487
✓ c50_SmartFactoryProduction.java	97.9 %	143	3	146
> c50_SmartFactoryProduction	89.7 %	26	3	29
> ProductionLine	100.0 %	117	0	117

4. Fuzz testing branch coverage: 70%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	0.8 %	7	835	842
✓ src	0.8 %	7	835	842
✓ code	0.8 %	7	835	842
✓ c50_SmartFactoryProduction.java	70.0 %	7	3	10
> c50_SmartFactoryProduction		0	0	0
> ProductionLine	70.0 %	7	3	10

5. Fuzz testing instruction coverage: 89.7%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	0.6 %	131	21,360	21,491
✓ src	0.6 %	131	21,360	21,491
✓ code	0.6 %	131	21,360	21,491
✓ c50_SmartFactoryProduction.java	89.7 %	131	15	146
> c50_SmartFactoryProduction	89.7 %	26	3	29
> ProductionLine	89.7 %	105	12	117

6. Z3 branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	1.2 %	10	832	842
✓ src	1.2 %	10	832	842
✓ code	1.2 %	10	832	842
✓ c50_SmartFactoryProduction.java	100.0 %	10	0	10
> c50_SmartFactoryProduction		0	0	0
> ProductionLine	100.0 %	10	0	10

7. Z3 instruction coverage: 97.9%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	0.7 %	143	21,344	21,487
✓ src	0.7 %	143	21,344	21,487
✓ code	0.7 %	143	21,344	21,487
✓ c50_SmartFactoryProduction.java	97.9 %	143	3	146
> c50_SmartFactoryProduction	89.7 %	26	3	29
> ProductionLine	100.0 %	117	0	117

8. Unimproved V-method branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	1.2 %	10	832	842
✓ src	1.2 %	10	832	842
✓ code	1.2 %	10	832	842
✓ c50_SmartFactoryProduction.java	100.0 %	10	0	10
> c50_SmartFactoryProduction		0	0	0
> ProductionLine	100.0 %	10	0	10

9. Unimproved V-method instruction coverage: 97.9%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	0.7 %	143	21,344	21,487
✓ src	0.7 %	143	21,344	21,487
✓ code	0.7 %	143	21,344	21,487
✓ c50_SmartFactoryProduction.java	97.9 %	143	3	146
> c50_SmartFactoryProduction	89.7 %	26	3	29
> ProductionLine	100.0 %	117	0	117